

Optimization And Genetic Algorithm

Dr. Thi My Chinh Chu

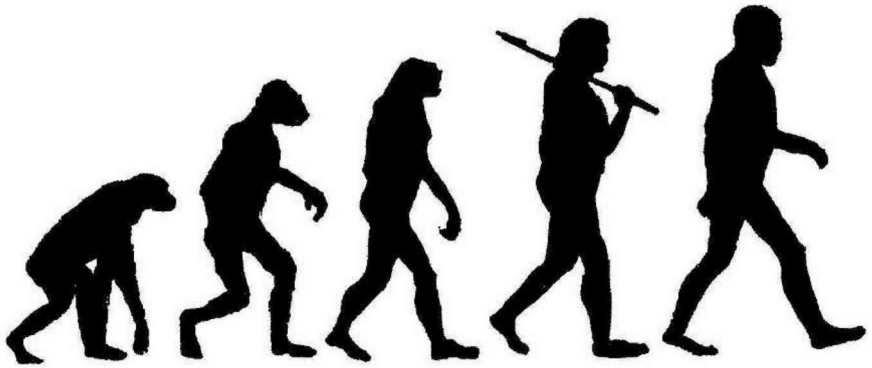
Dr. Huseyin Kusetogullari

Simulation of natural evolution

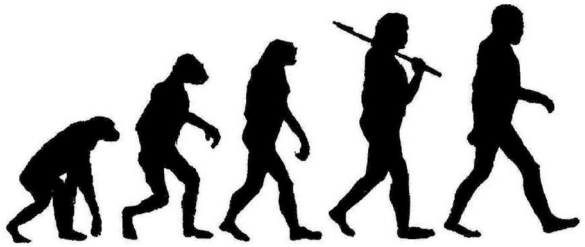
- On 1 July 1858, Charles Darwin presented his theory of evolution before the Linnean Society of London.

- This day marks the beginning of a revolution in biology.

- Darwin's classical theory of evolution, together with Weismann's theory of natural selection and Mendel's concept of genetics, now represent the neo-Darwinian paradigm.



Evolutionary Computing



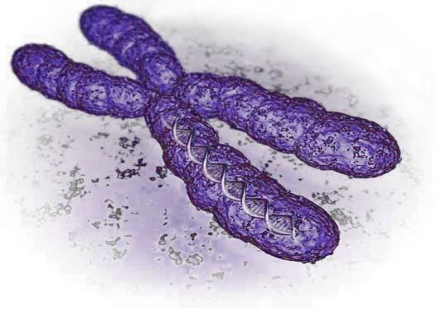
Humans have evolved (and continue to evolve) over time….

We presume that prehistoric species of man were less intelligent than we are

As conditions on earth change, species become extinct (e.g., the dinosaurs) and other species emerge

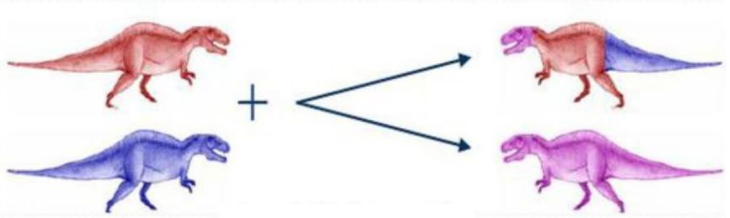
Can we model this process and use ”evolve” strategy in new programs for optimization

Biological Background

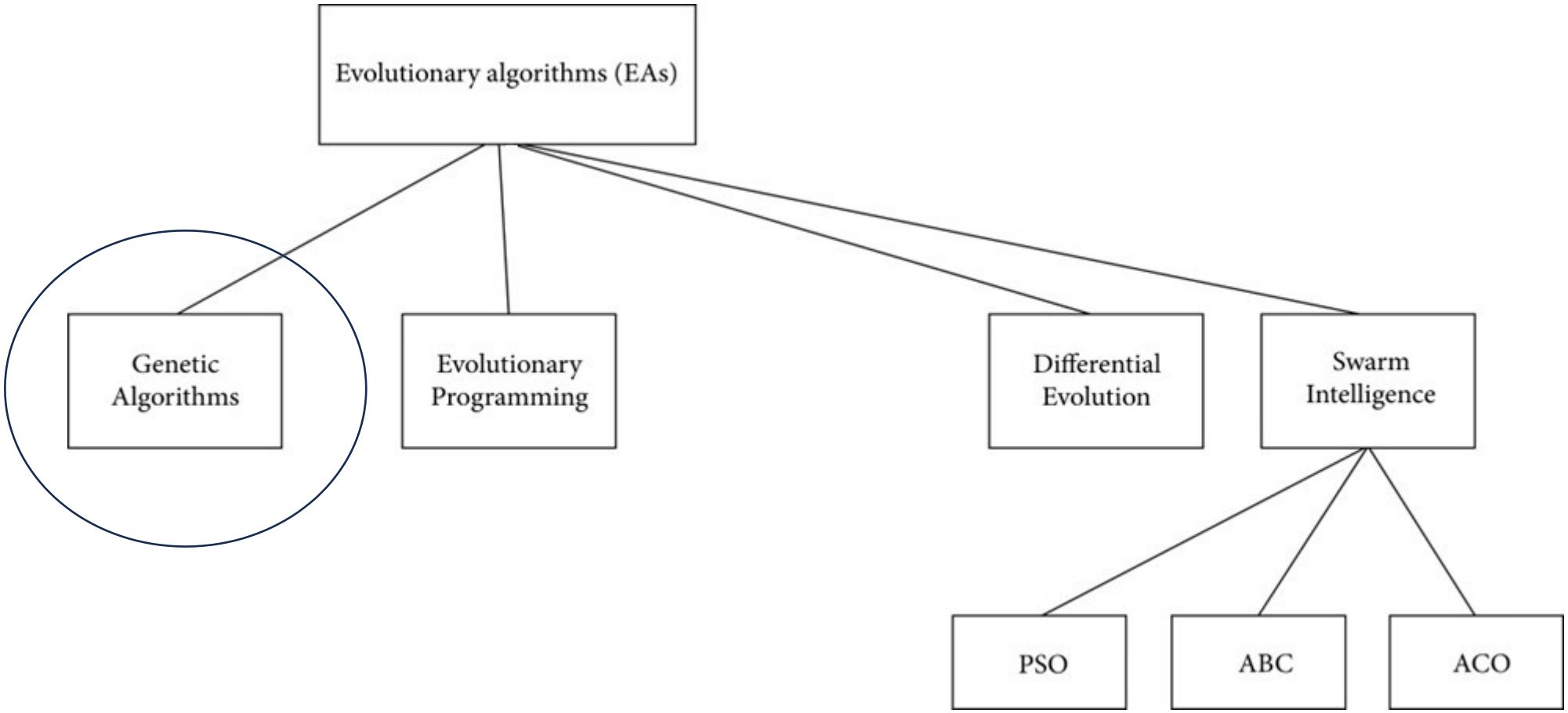


- All Living organisms consists of cells.
- Each cell has a set of **Chromosomes**.
- Chromosomes are strings of DNA and consists of **genes.**
- Genes are blocks of DNA.
  Each gene encodes a **trait**, for example color of eyes.
- **Reproduction**
- Organisms produce an offspring (children) similar to themselves but can have variations (changes in traits) due to:
  ✓Mutations(random changes)
  ✓Crossover(offspring have combinations of features inherited from each parent)





Evolutionary Algorithms



Genetic algorithms Quick Overview

- Developed: in the 1970’s
- Early names: J. Holland, K. DeJong, D. Goldberg
- Typically applied to:
  - discrete optimization
- Attributed features:
  - not too fast
  - good heuristic for optimization problems
- Special Features:
  - Traditionally emphasizes combining information from good parents (crossover)
  - many variants, e.g., reproduction models, operators

Genetic Algorithm

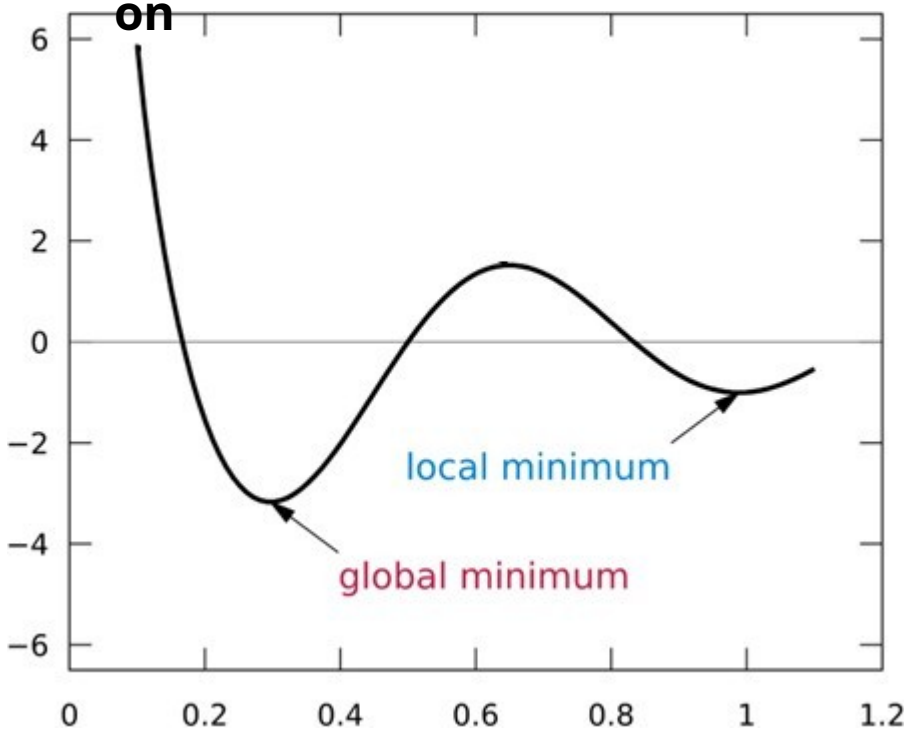
- It is a search-based optimization technique based on the principles of Genetics and Natural Selection.
- It is frequently used to find optimal or near-optimal solutions for the NP complex problems.
- It helps to decrease the time to search a solution.

Genetic Algorithm: Optimization

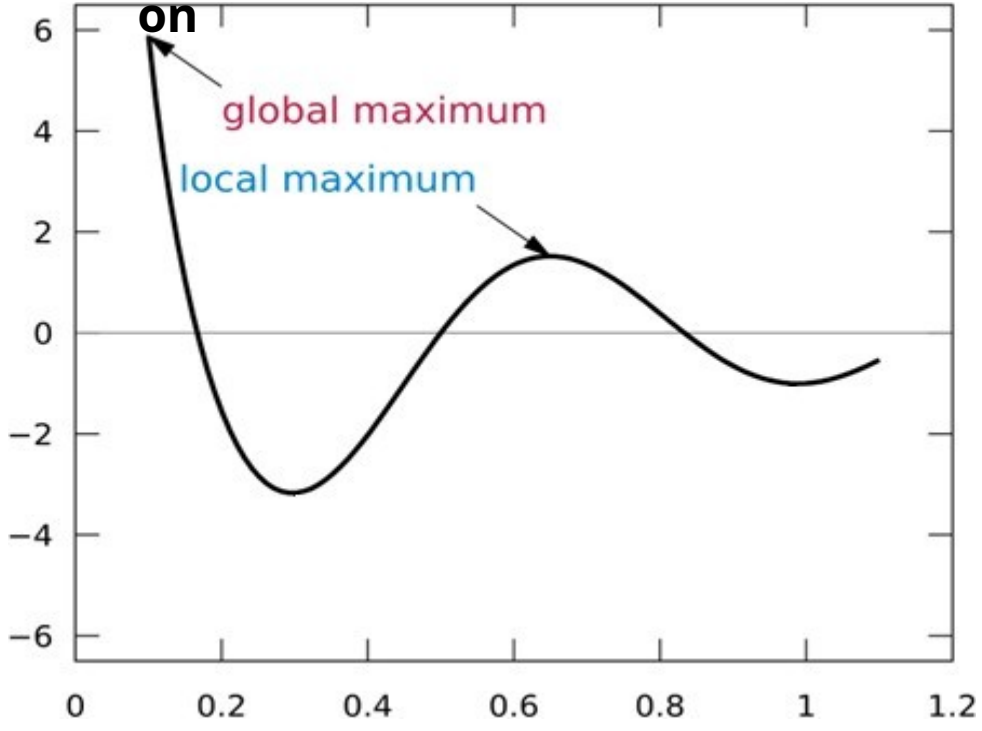
- Optimization problem is the problem of finding the best solution from all feasible solutions.
- Optimization Problem: computational problem where we try to maximize or minimize some value

Genetic Algorithm: Optimization

Minimization



Maximization



Used mainly in

- Finance
- Economy
- Computer Vision
- Pattern Recognition
- Medical
- Communications
- Scheduling
- Planning
- Routing
- Autonomous cars
- Many more...

Summary: Genetic Algorithms

A biological based method

Based on the Darwin’s theory (Evolution of

species) Based on **- survival of the fittest**

individual

Key steps: *reproduction, survive*

Try to simulate life.

Individual = solution

Environment =

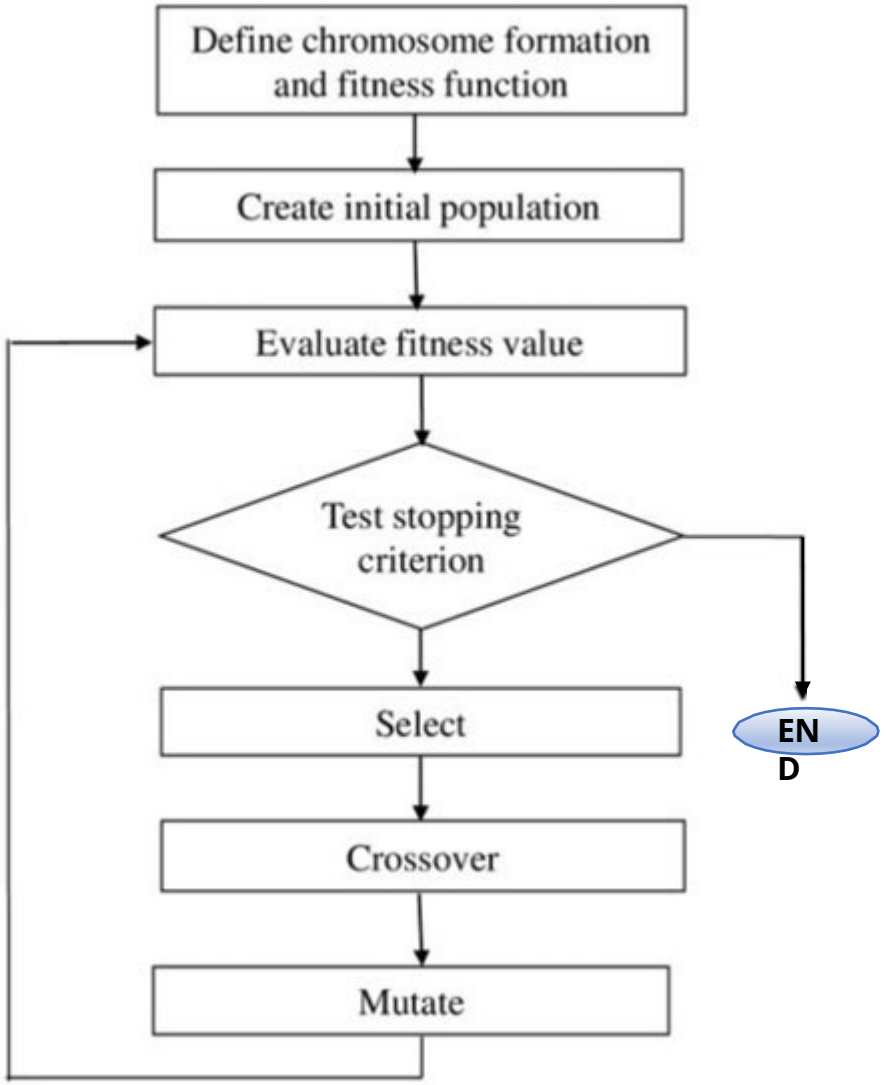
Components of a GA

A problem to solve, and …

- Encoding technique (*gene, chromosome*)
- Initialization procedure (*creation*), Random ‘guesses’ of the solution to an *optimization problem*
- Evaluation function (*environment*), calculates how good a guess solution is (*selection*)
- Genetic operators (*crossover, mutation*)
  - An operator for mixing good solutions to produce better ones – Crossover.
  - An operator to introduce diversity within the population – Mutation.
- Parameter settings (*practice and art*)

Genetic Algorithm

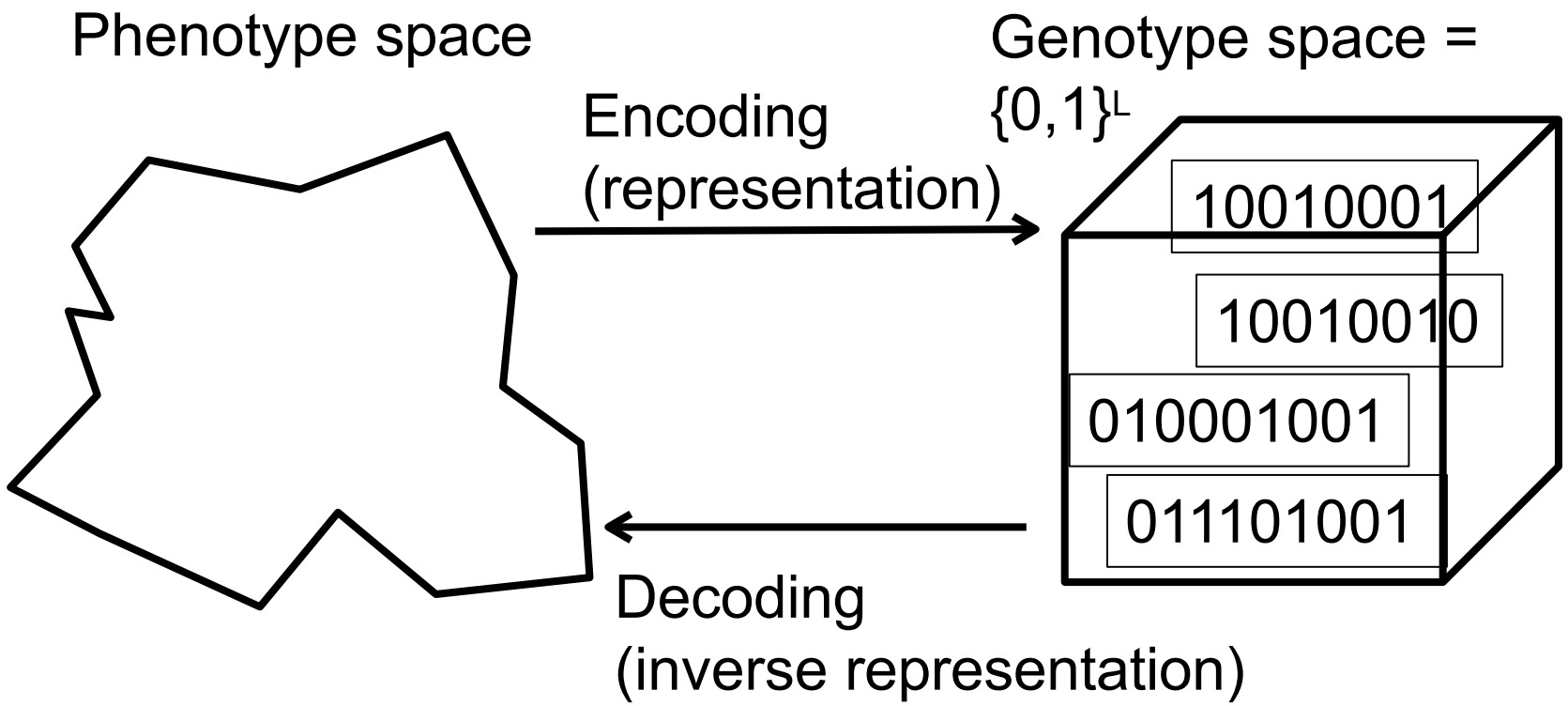
```
{
    define fitness
    function; initialize
    population;
    evaluate
    population;
    while
    TerminationCriteria
    NotSatisfied
    {
        perform selection technique;
        perform Crossover and
        Mutation; new population;
        evaluate population;
```



Genetic algorithms

Representation	Binary strings
Recombination	N-point or uniform
Mutation	Bitwise bit-flipping with fixed probability
Parent selection	Fitness
Survivor selection	All or some children replace parents
Speciality	Emphasis on crossover

Representation



Initial population

- Consist of number of chromosomes or solutions.
- Generated randomly.
- One time process.
- It can be with a set of binary numbers, real numbers, etc.

1	0	1	1	0	1	0	0	1	0	1	1	0	1	0	0

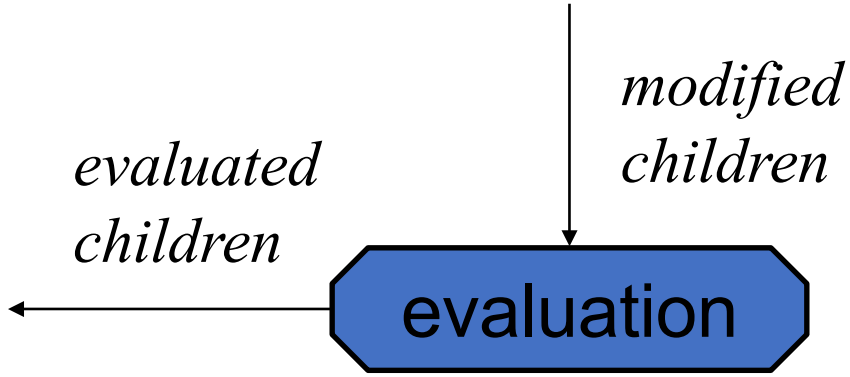
A 16-bit binary string of an artificial chromosome

Fitness function

- **Definition**: Fitness Function (also known as the Evaluation Function) evaluates how close a given solution is to the optimum solution of the desired problem.
- The fitness function should quantitatively measure how fit a given solution is in solving the problem.

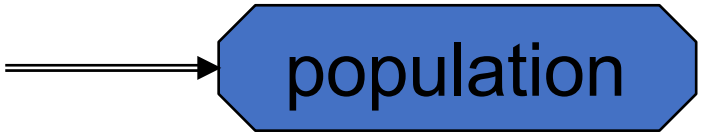
- The fitness function should be clearly defined.
- Should be able to understandable how the fitness score is calculated.
- Each problem has its own fitness function.
- The fitness function should be developed based on a given problem.

Evaluation



- The evaluator decodes a chromosome and assigns it a fitness measure.
- The evaluator is the only link that provides evaluation of each chromosome or child.

Populati
on



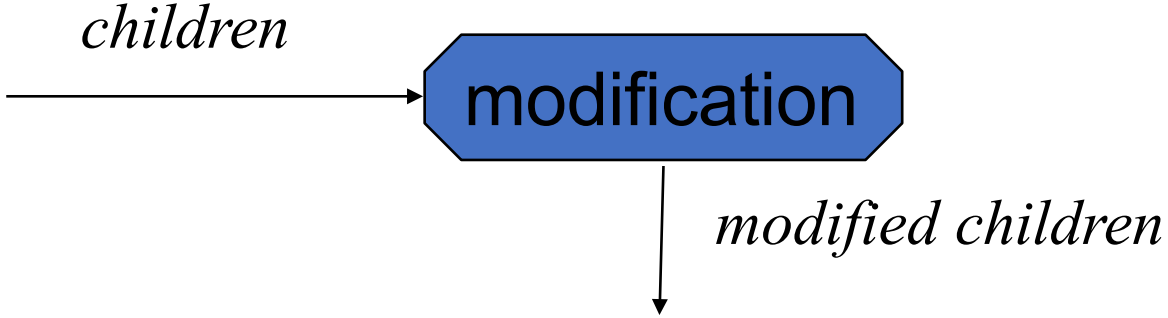
Chromosomes could be:

- Bit strings
- Real numbers

(0101 ... 1100)

(43.2 -33.1 ... 0.0 89.2)

Chromosome Modification: Reproduction



- Modifications are stochastically triggered
- Operator types are:
  - Mutation
  - Crossover (recombination)

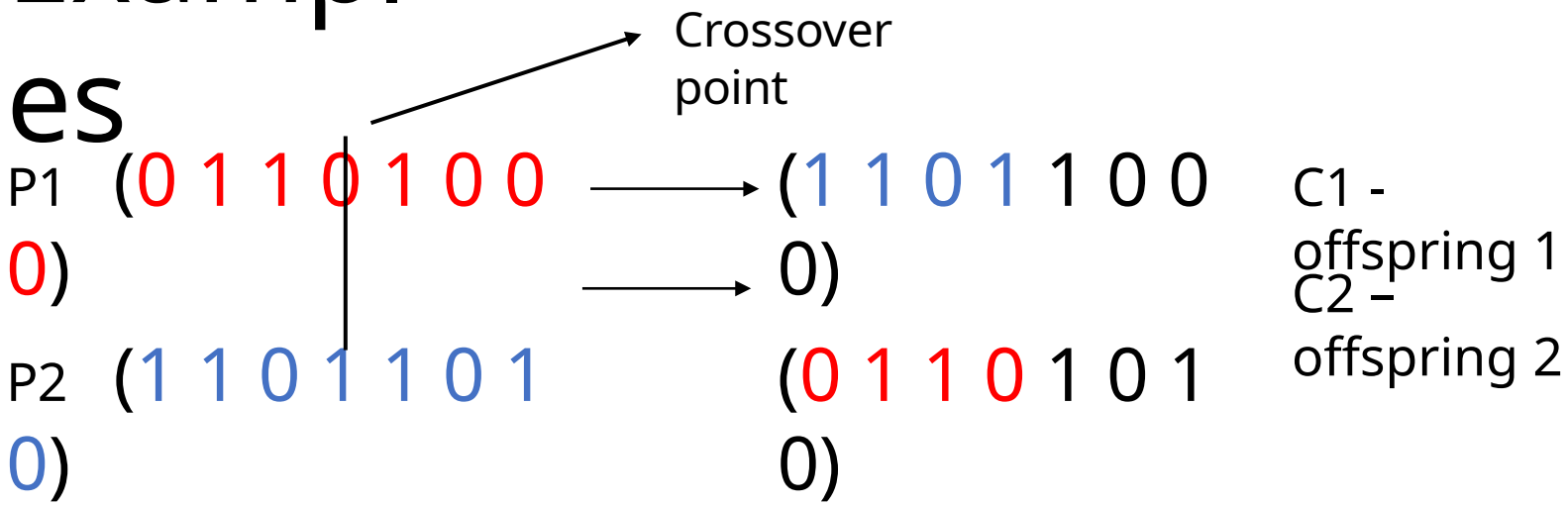
Crossover: Single-point

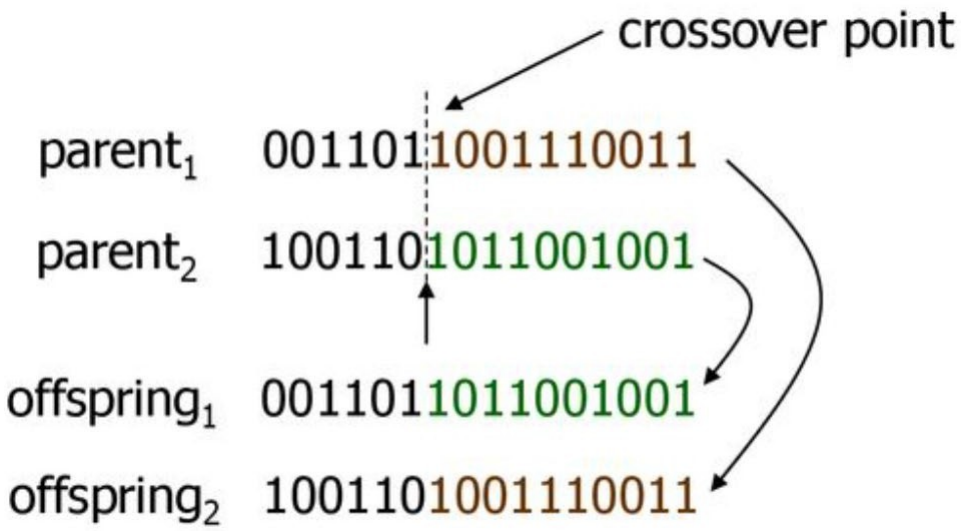
Crossover is a critical feature of

genetic algorithms:

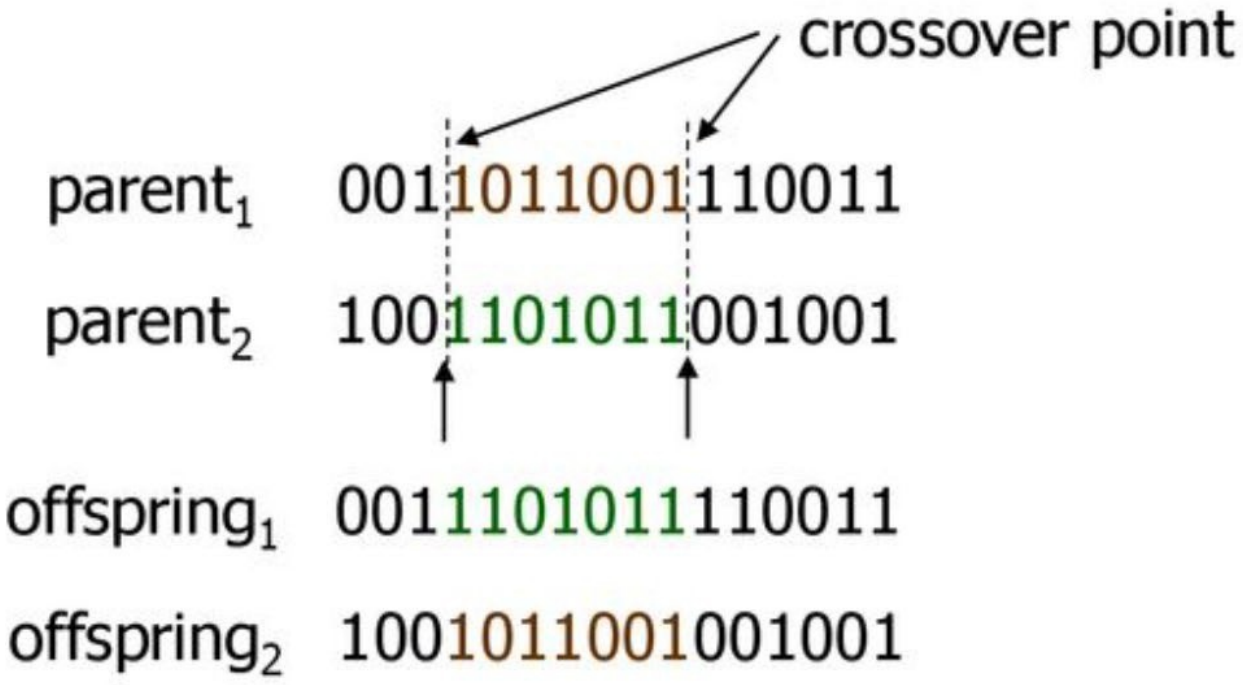
- It greatly accelerates search early in evolution of a population
- It leads to effective combination of schemata (subsolutions on different chromosomes)

Exampl
es


Crossover point
P1 (0 1 1 0 1 0 0 0)
P2 (1 1 0 1 1 0 1 0)
(1 1 0 1 1 0 0 0)
(0 1 1 0 1 0 1 0)
C1 - offspring 1
C2 – offspring 2

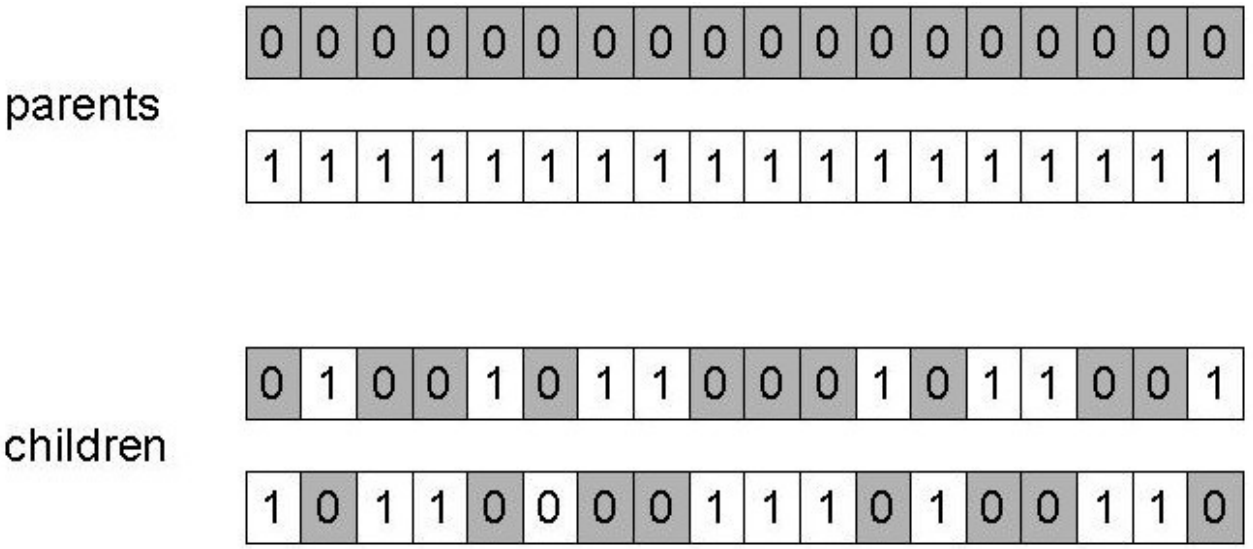

crossover point
parent1 0011011001110011
parent2 1001101011001001
offspring1 0011011011001001
offspring2 1001101001110011

Crossover: two-point



Uniform crossover

- Assign 'heads' to one parent, 'tails' to the other
- Flip a coin for each gene of the first child
- Make an inverse copy of the gene for the second child
- Inheritance is independent of position



Mutation: Local Modification

Before: (1 0 1 1 0 1 1 0)

After: (1 0 1 0 0 1 1 0)

Before: (1.38 -69.4 326.44 0.1)

After: (1.38 -67.5 326.44 0.1)

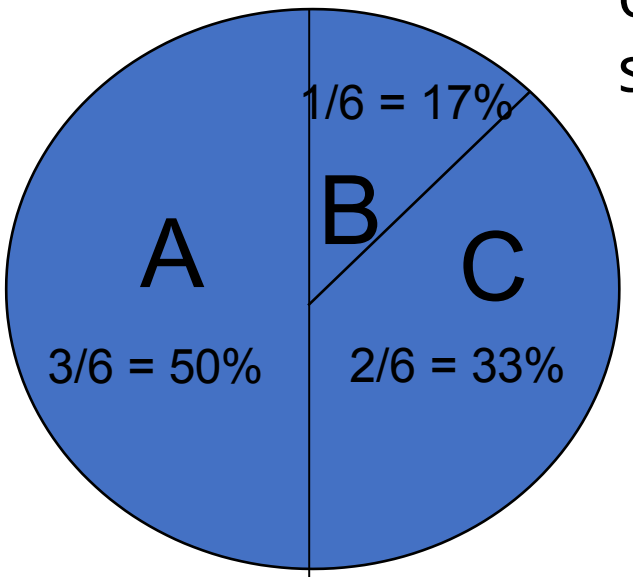
- Causes movement in the search space (local or global)
- Restores lost information to the population

Mutation

- Operation on temporary offspring from crossover
- Escape from the local minimum and explore other zone.
- Avoid pre-maturity
- Low mutation rate (typically 1%～20%)
- For binary GA, the mutated bit will be flip. (1→0, 0→1)
- For continuous parameter GA, the mutated gene will be replaced by an arbitrary value.

Selection: Roulette Wheel Selection

- Main idea: better individuals get higher chance
  - Chances proportional to fitness
  - Implementation: roulette wheel technique
    - Assign to each individual a part of the roulette wheel
    - Spin the wheel n times to select n individuals

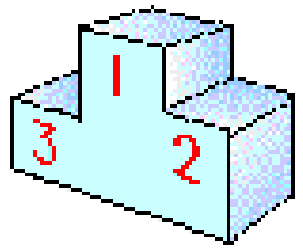


fitness(A) = 3

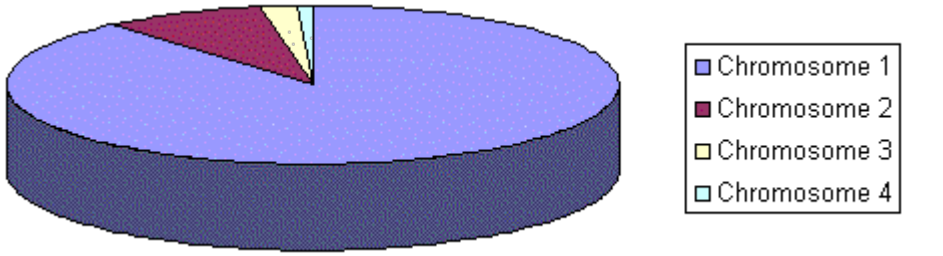
fitness(B) = 1

fitness(C) = 2

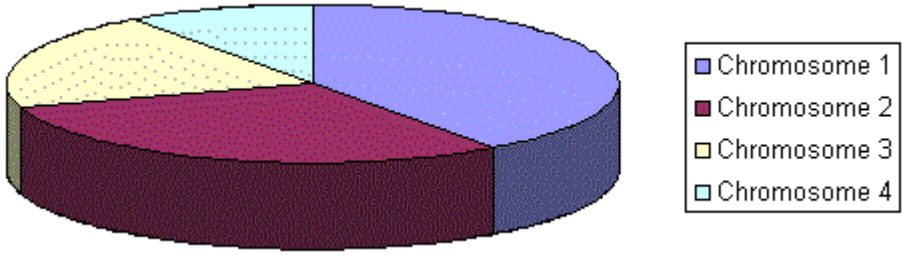
Rank Selection



- All individuals are sorted according to their fitness.
- Each individual is then assigned a probability of being selected from some prior probability density. (The worst will have fitness 1, second worst 2 etc. and the best will have fitness N (number of chromosomes in population).)



Situation before ranking (graph of fitnesses)



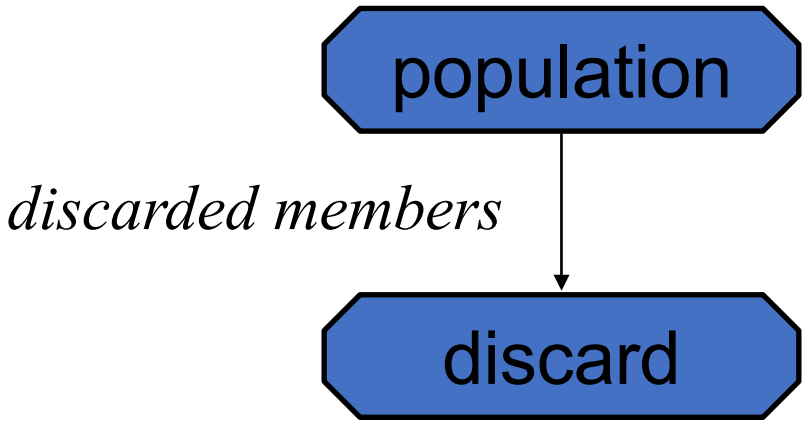
Situation after ranking (graph of order fitnesses)

*** After this all the chromosomes have a chance to be selected. But this method can lead to slower convergence, because the best chromosomes do not differ so much from other ones.**

Cont.

- Steady State Selection: keep most of the existing population and replace only a few better individuals as a parent
- Tournament Selection: randomly choose a small group of chromosomes, compare their fitness, and select the best one
- Elitism Selection: make sure that the best few chromosomes is carried over to the next generation
- Others: there are other many selection strategies, depending on the problem and the genetic algorithm being used

Deletion



- Entire or some individuals in populations replaced with each iteration
- A few members replaced each generation

Using Genetic Algorithm to Solve Optimization Problems

A Simple Example

The Traveling Salesman Problem (TSP):

Find a tour of a given set of cities so that

- each city is visited only once
- the total distance traveled is minimized

Representation

Representation is an ordered list

of city numbers known as an

order-based GA.

1) London	3) Dunedin	5) Beijing	7) Tokyo	
2) Venice	4) Singapore	6) Phoenix	8) Victoria	

CityList1 (3 5 7 2 1 6 4 8)

CityList2 (2 5 7 6 8 1 3 4)

Crossover

Crossover combines
inversion and recombination:

Parent1	(3	5	7	2	1	6	4	8)
Parent2	(2	5	7	6	8	1	3	4)

Child1	(3	5	7	2	1	6	3	4)
Child2	(2	5	7	6	8	1	4	8)

This is one point crossover
operator.

Permutation Crossover in GA for TSP

- Goal: Create a new by combining two parent tours without repeating or losing any city.
- Permutation crossover preserves a valid tour for TSP, such as, every city appears exactly once, no duplicate cities, no missing cities

Crossover	Main Idea	Simple Description
1. Partially Mapped Crossover (PMX)	Exchange a segment	Copies a section from one parent and uses a mapping to avoid duplicate cities.
2. Order Crossover (OX)	Preserve city order	Copies a segment, then fills the remaining cities according to their order in the other parent.

An Example of Partially Mapped Crossover

- Parents: Parent 1: (1 2 3 | 4 5 6 | 7 8 9)

  Parent 2: (9 3 8 | 1 2 7 | 4 5 6)

- Crossover Points: Choose two random cut points (marked by |)

- Swap Segments: The middle segment of Parent 1 (4 5 6) goes into Offspring 1, and the middle segment of Parent 2 (1 2 7) goes into Offspring 2.

  - Child 1: 9 3 8 | 4 5 6 |4 5 6)

  - Child 2: (1 2 3 | 1 2 7 | 7 8 9)

- Resolve Mappings: Map the swapped numbers to fix conflicts (duplicates) using the correspondence between the two middle segments (4↔1, 5↔2, 6↔7).

- Final Offsprings:

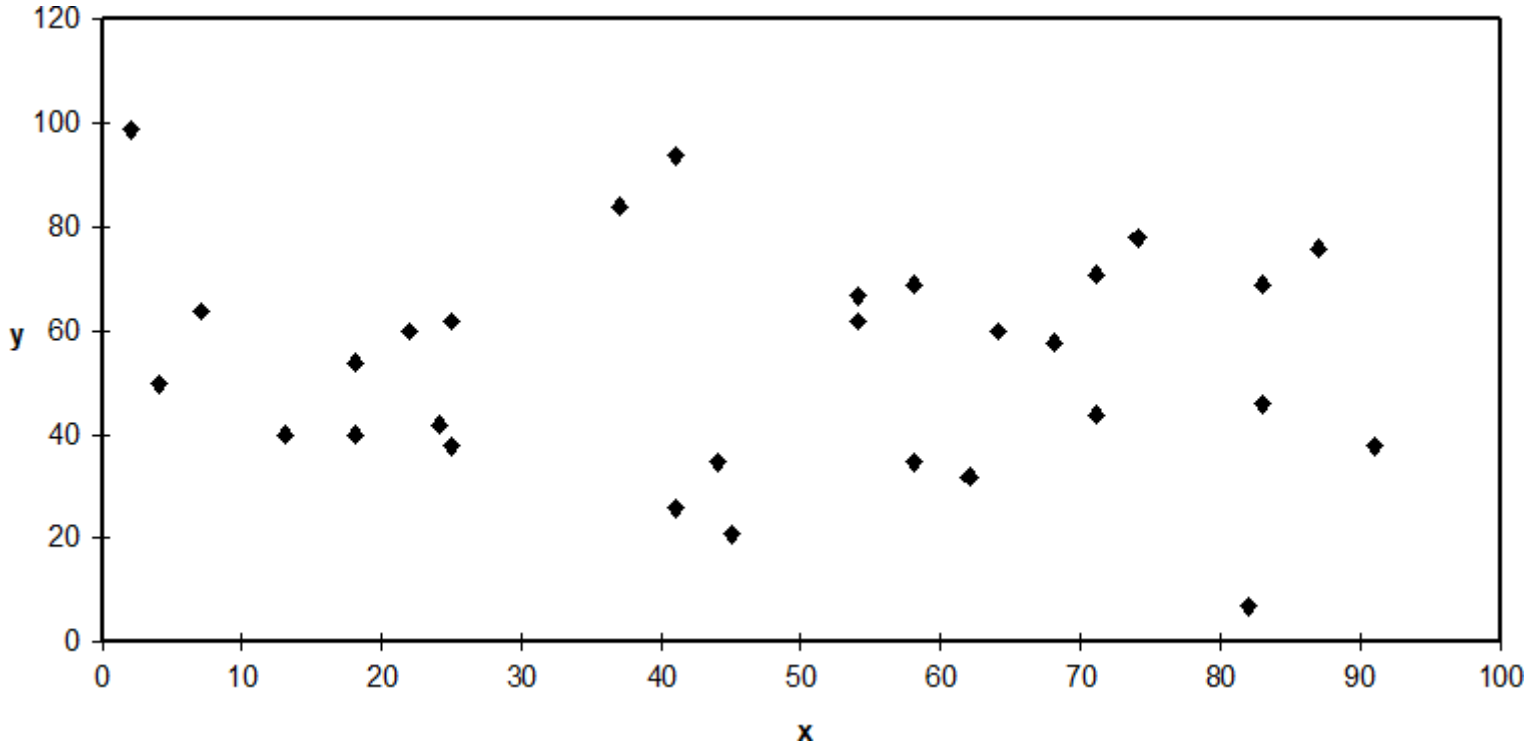
  - Child 1: (9 3 8| 4 5 6 |1 2 7)

  - Child 2: (4 5 3| 1 2 7 |6 8 9)

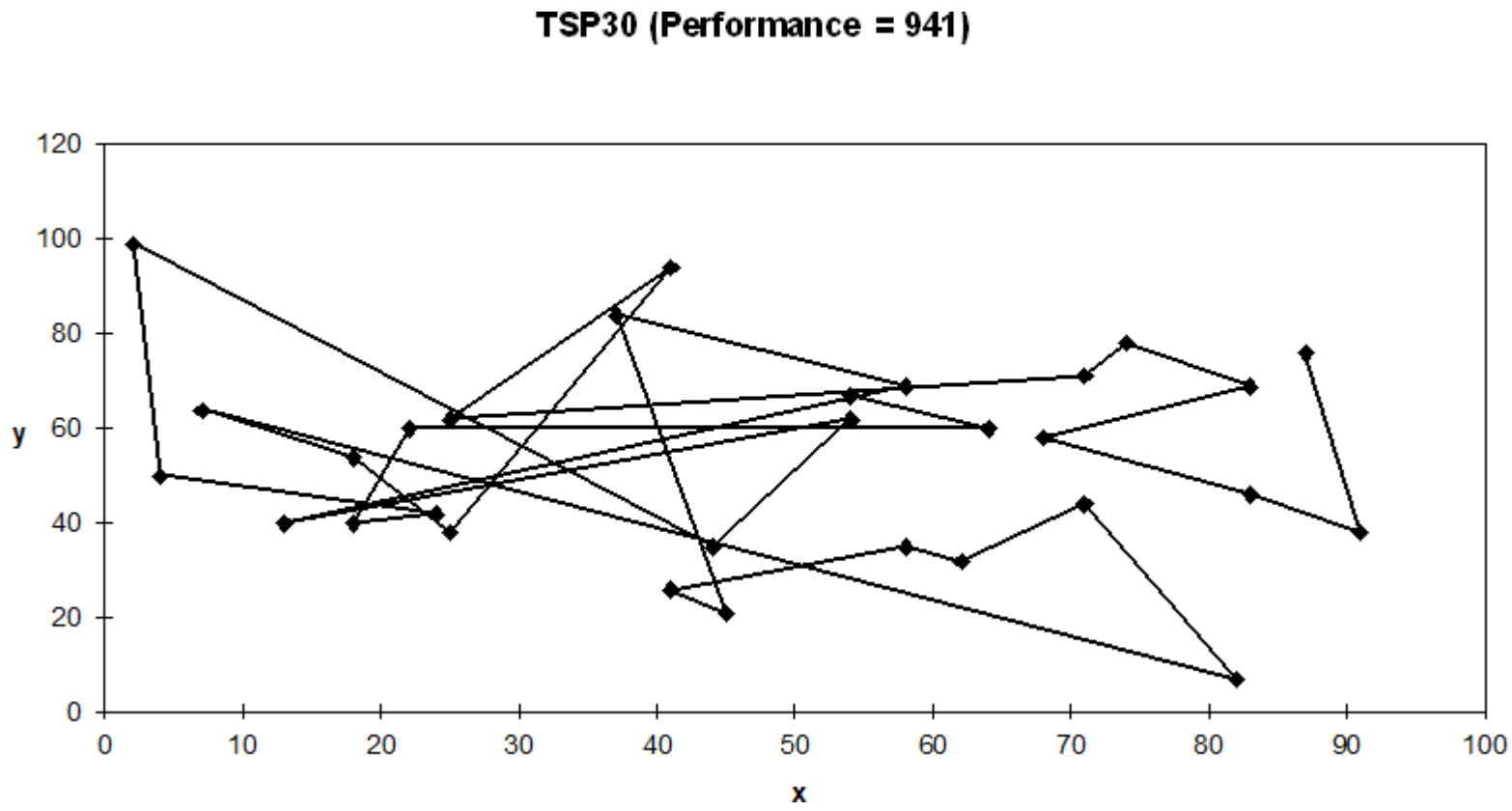
An Example of Order Crossover

- Parents:
  - Parent 1: (1 2 3 | 4 5 6 | 7 8 9)
  - Parent 2: (9 3 8 | 1 2 7 | 4 5 6)
- Crossover Points: Choose two random cut points (marked by |)
- Swap Segments: The middle segment of Parent 1 (4 5 6) goes into Offspring 1, and the middle segment of Parent 2 (1 2 7) goes into Offspring 2.
  - Offspring 1: (- - -| 4 5 6 |- - -)
  - Offspring 2: (- - - | 1 2 7 |- - -)
- Fill the remaining empty positions in the offspring using the order of cities as they appear in the other Parent
  - Offspring 1: (9 3 8| 4 5 6 |1 2 7)
  - Offspring 2: (3 4 5| 1 2 7 |6 8 9)

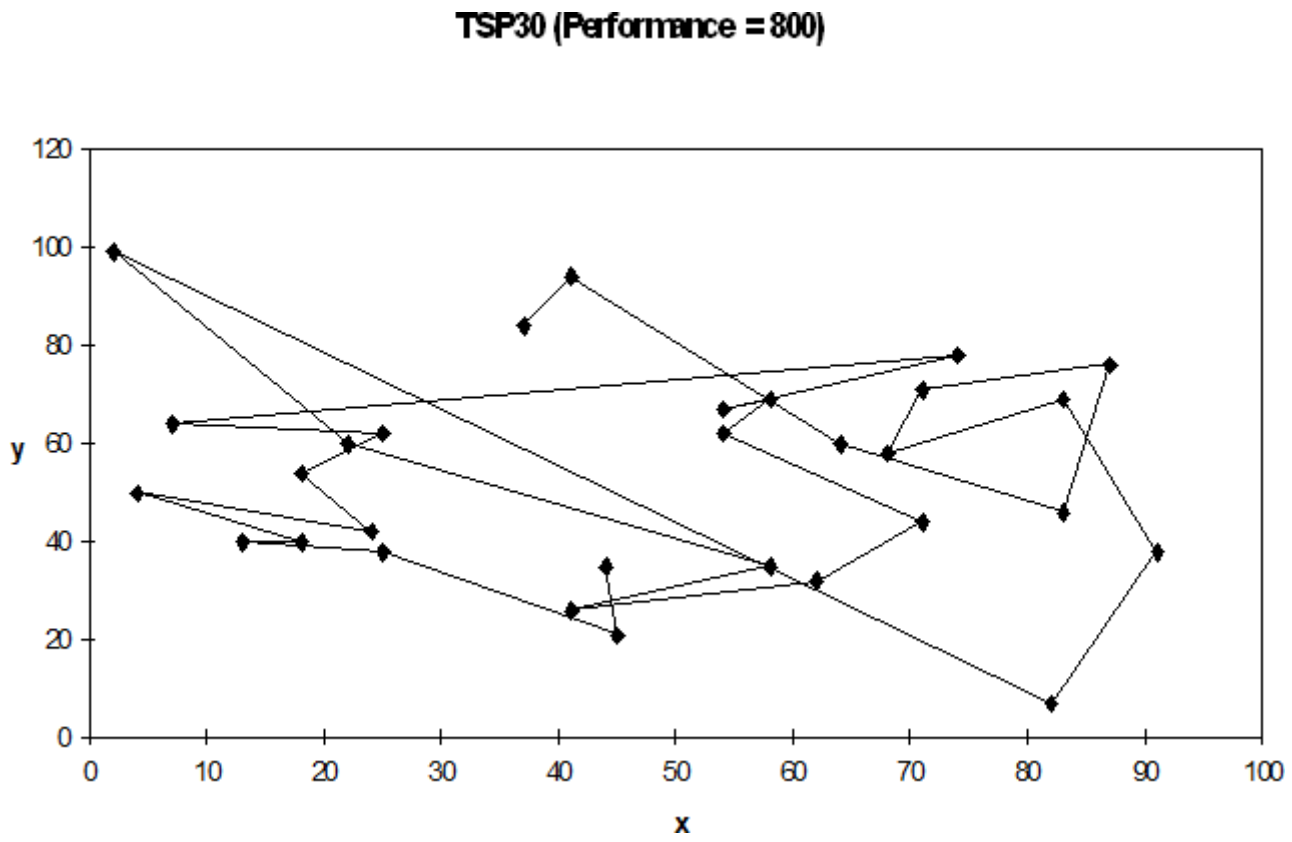
TSP Example: 30 Cities



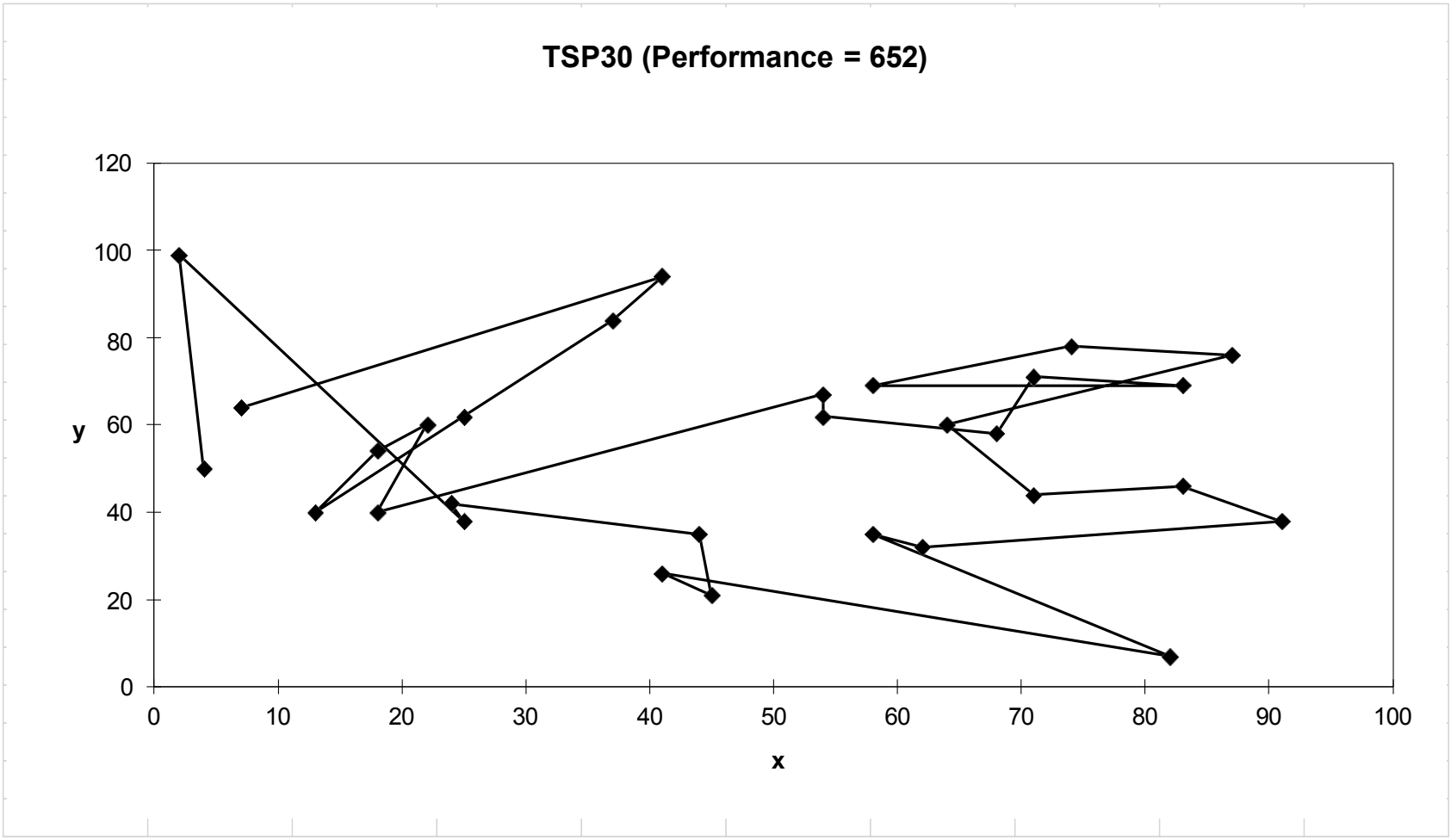
Solution (Distance = 941)



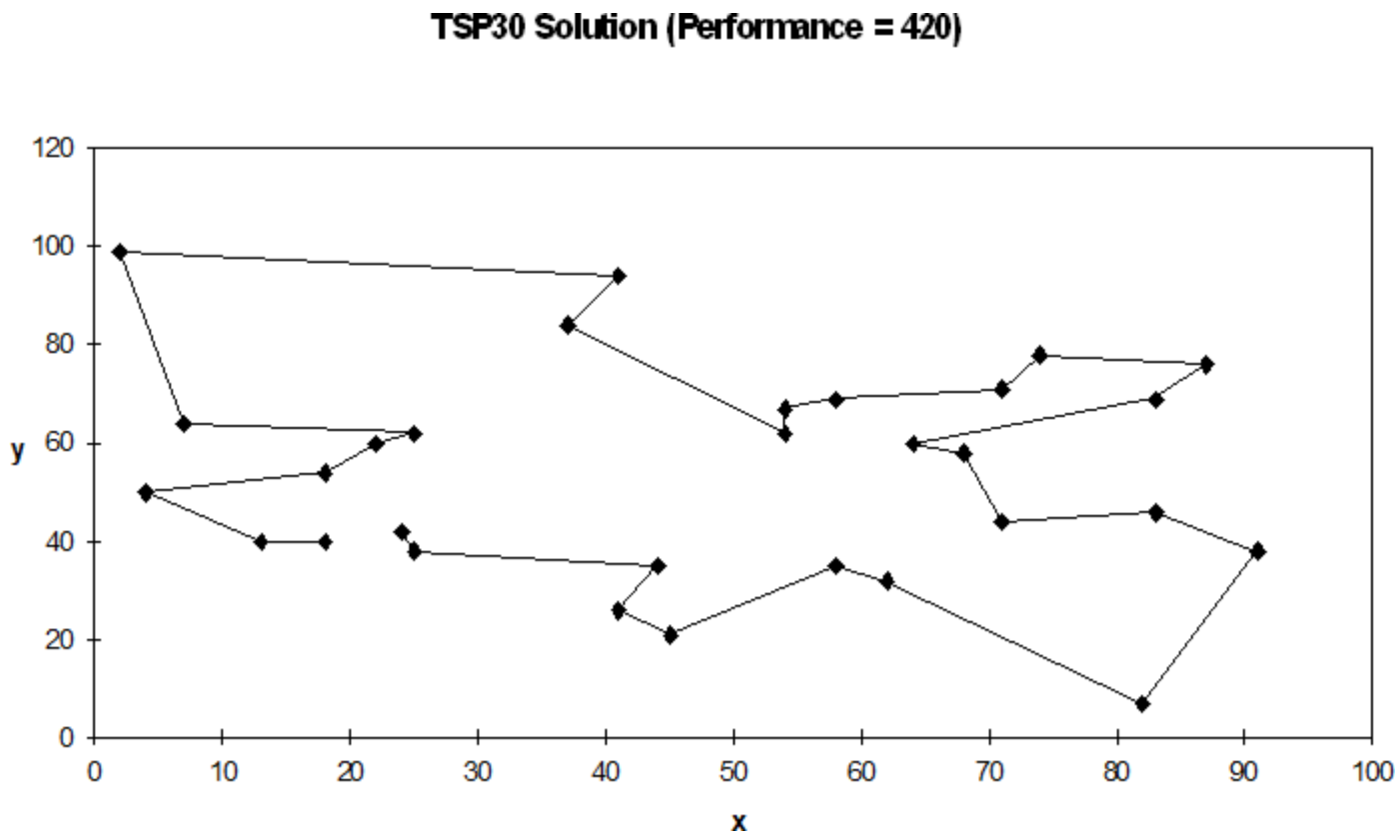
Solution (Distance = 800)



Solution (Distance = 652)



Best Solution (Distance = 420)



Example 2 (Negnevitsky, 2002)

- To find the maximum value of the function $(15x-x^2)$, where parameter x varies between 0 and 15. Assumption: x takes only integer values.

Integer	Binary code
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Example (Contd.)

- Suppose that the size of the chromosome population N is 6, the crossover probability p_c equals 0.7 and the mutation probability p_m equals 0.001, the fitness function in our example is defined by

$$f(x) = (15x-x^2)$$

Example (Contd.)

- The GA creates an initial population of chromosomes by filling six 4-bit strings with randomly generated ones and zeros.

Chromosome label	Chromosome string	Decoded integer	Chromosome fitness	Fitness ratio %
X1	1100	12	36	16.5
X2	0100	4	44	20.2
X3	0001	1	14	6.4
X4	1110	14	14	6.4
X5	0111	7	56	25.7
X6	1001	9	54	24.8

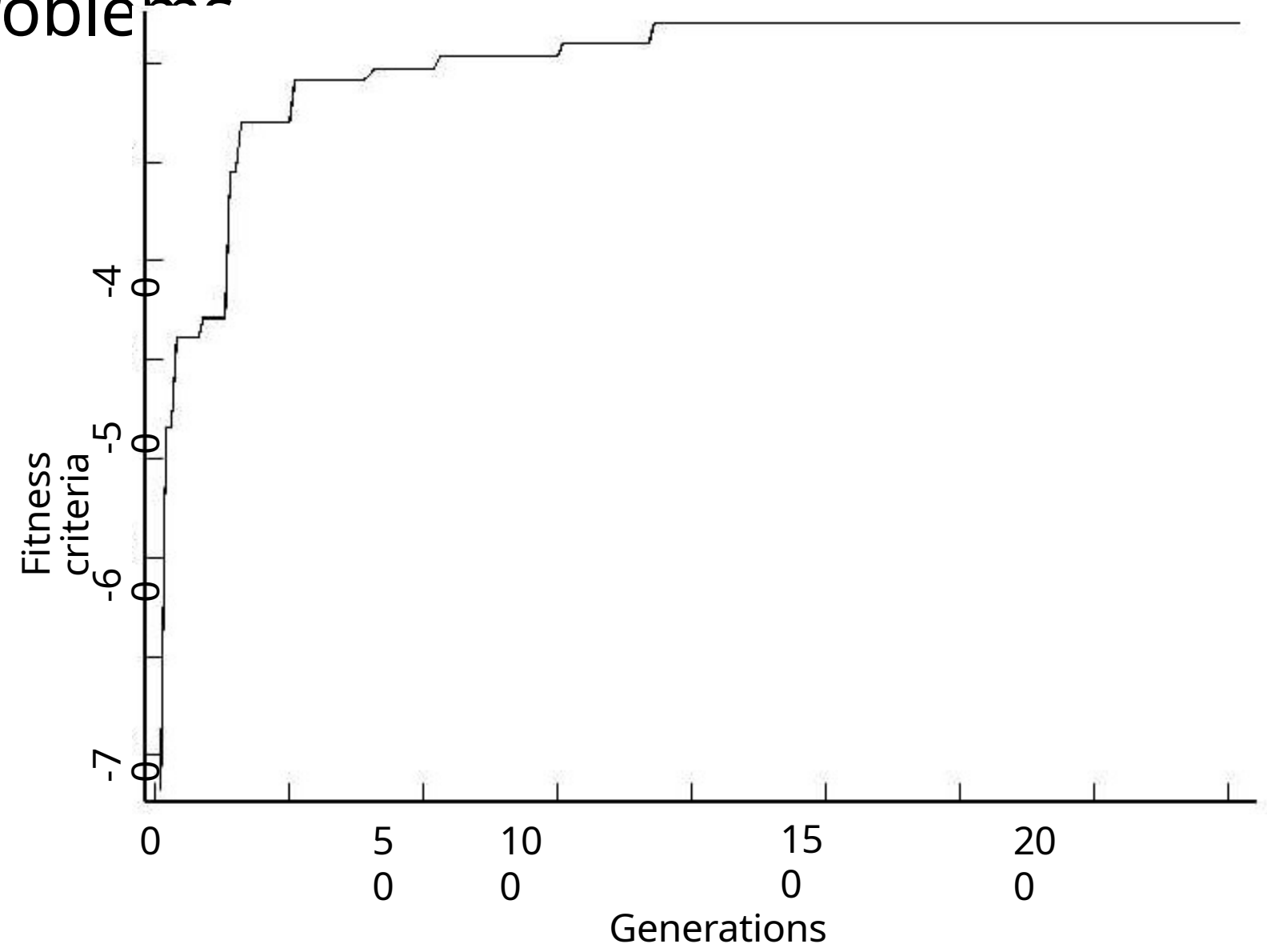
Example (Cont.)

- Last column in the previous table shows the ratio of the individual chromosome fitness to the populations total fitness.

- According to the example, chromosomes X5 and X6 have a pretty good chance of being selected for mating, while X3 and X4 have a poor chance.

- Roulette selection wheel is one of the the most commonly used chromosome selection technique.

Genetic Algorithm: Maximization problems

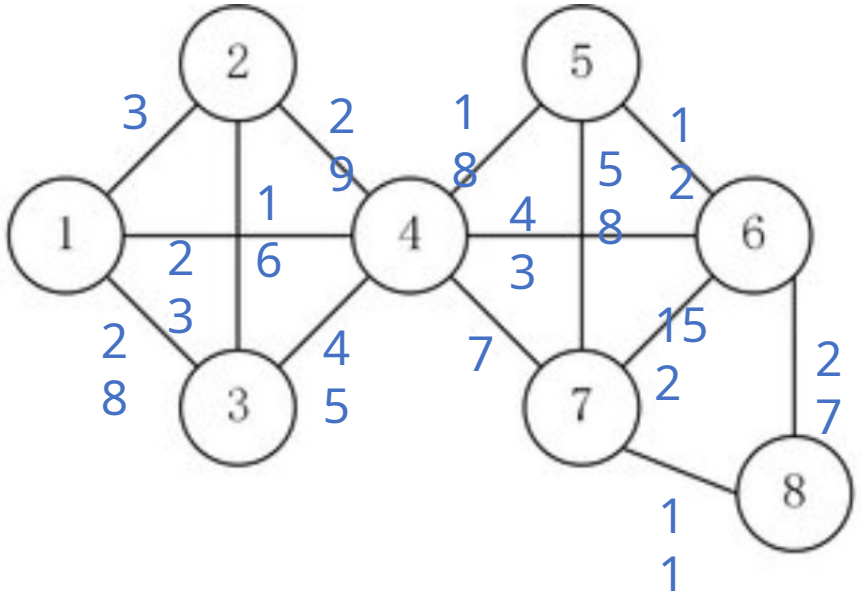


Network routing problem

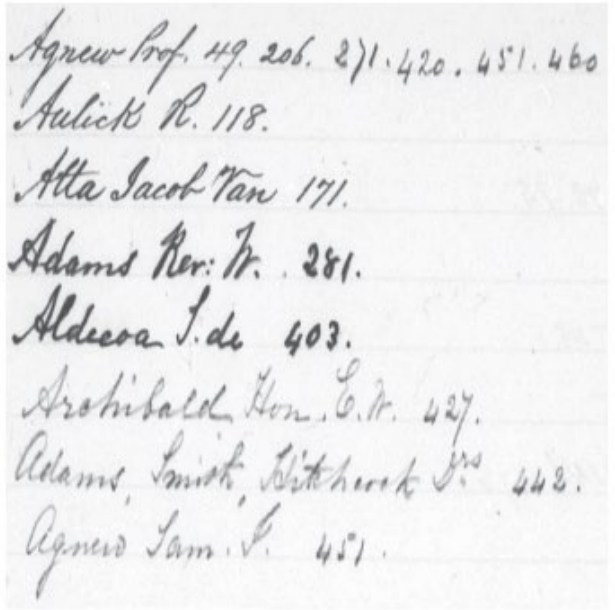
Consider a network model with 8 nodes.

Find the shortest path between the starting node 1 and destination node 8 using GA. Minimization or Maximization?

Number of genes in a chromosome? Binary or real numbers? How to define a fitness function?



Example: Image application



300

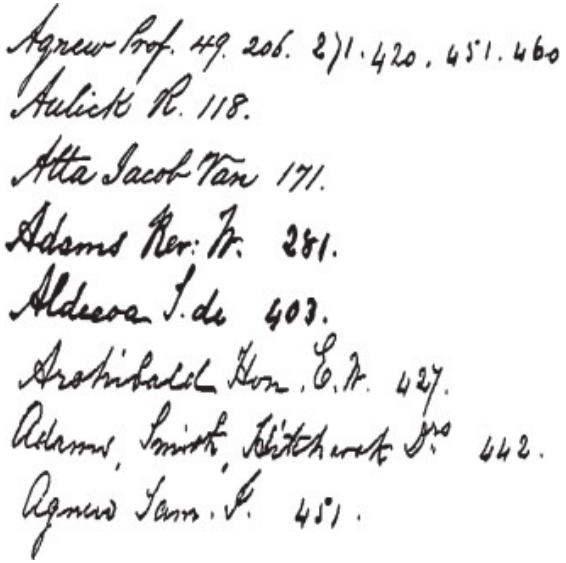
200

Input image

How to design? parameters?

Genetic Algorithm

300



200

Target image

Minimization or Maximization problem?

Number of genes in a chromosome? Binary or real numbers? How to define a fitness function?

Genetic Algorithm Hyper-parameters

- Genetic Algorithm has several parameters that can be tuned to obtain better results. The set of parameters is called Hyper-parameters. The most important hyper-parameters are:

- **Population size**: number of chromosomes in the population. This parameter impacts the algorithm running time. The more chromosomes in the population the longer the algorithm will take.

- **Crossover rate**: percentage of chromosomes selected from generation *t* to be used in generation *t+1* for reproduction.

- **Mutation rate**: probability that a chromosome mutates.

- **Generation number**: number of iterations of the algorithm before it stops. The running time is also directly impacted by this parameter.

Issues for GA Practitioners

- Choosing basic implementation issues:
  - representation
  - population size, mutation rate, …
  - selection, deletion policies
  - crossover, mutation operators
- Termination Criteria
- Performance, scalability
- Solution is only as good as the evaluation function (often hardest part)

Benefits of Genetic Algorithms

- Concept is easy to understand
- Modular, separate from application
- Supports multi-objective optimization
- Good for “noisy” environments
- Always an answer; answer gets better with time
- Inherently parallel; easily distributed

Benefits of Genetic Algorithms (cont.)

- Many ways to speed up and improve a GA-based application as knowledge about problem domain is gained
- Easy to exploit previous or alternate solutions
- Flexible building blocks for hybrid applications
- Substantial history and range of use

When to Use a GA

- Alternate solutions are too slow or overly complicated
- Need an exploratory tool to examine new approaches
- Problem is similar to one that has already been successfully solved by using a GA
- Want to hybridize with an existing solution
- Benefits of the GA technology meet key problem requirements

Thank you for listening